ResolveIQ

Technical Architecture Document

AI-Powered Frictionless Dispute & Chargeback Resolution · Round 1 Submission · 2025

Version 1.0

1. Document Overview

This document describes the complete technical architecture of ResolveIQ — an AI-powered dispute resolution platform designed for card issuers like American Express. It covers system design, component interactions, data flows, API contracts, security model, and infrastructure.

> **ResolveIQ resolves 70%+ of disputes automatically in under 5 minutes — replacing a 14–45 day manual process with a three-layer AI pipeline: Evidence Autopilot → FairScore™ → Explainability Engine.**

1.1 Design Principles

- **Zero-touch by default:** Evidence is auto-collected from all available sources; manual input supplements, never starts from scratch
- **Transparent by design:** Every decision is structurally explainable — not post-hoc rationalised
- **Dual-perspective fairness:** Both card member and merchant evidence are explicitly modelled and scored
- **Graceful degradation:** System continues with available data if any source fails; never blocks resolution
- **Human-in-the-loop safety:** Low-confidence cases route to human review automatically — safe to deploy at any accuracy level
- **Immutable audit trail:** Every action, decision, and evidence item is hash-chained and permanently logged

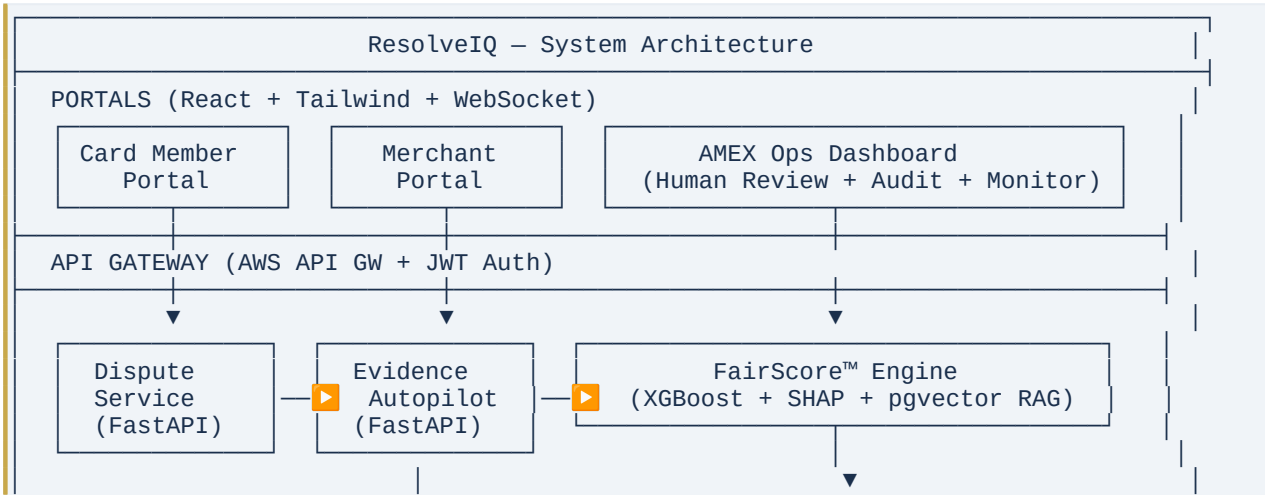
2. System Architecture

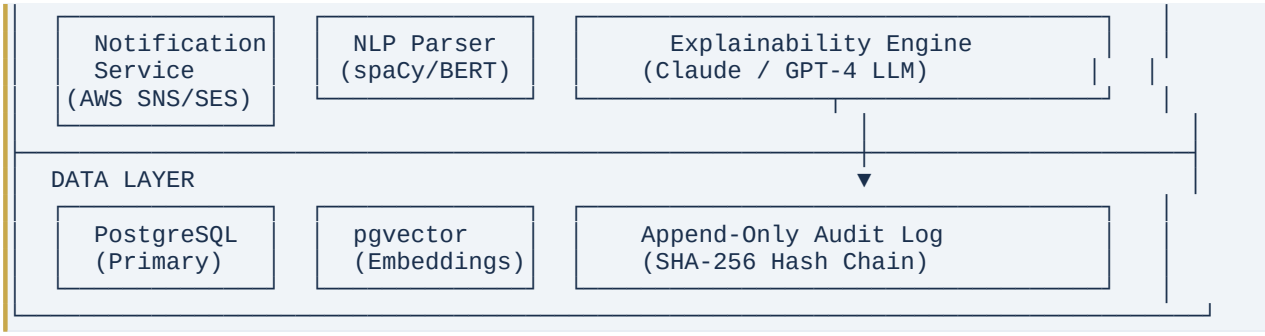
2.1 High-Level Architecture

ResolveIQ is structured as a set of independently deployable microservices, each corresponding to a distinct stage of the dispute resolution pipeline. Services communicate via REST APIs with async event queues for non-blocking operations.

> Architecture style: Event-driven microservices on AWS serverless infrastructure. Each layer is independently scalable, testable, and deployable without affecting adjacent layers.

Pipeline overview (left to right):





2.2 Service Inventory

Service	Technology	Responsibility
Dispute Service	FastAPI (Python)	Receives dispute filings, orchestrates pipeline, manages dispute lifecycle state machine
Evidence Autopilot	FastAPI (Python)	Auto-collects transaction data, merchant webhooks, email evidence; returns structured evidence bundle
NLP Evidence Parser	spaCy + DistilBERT	Entity extraction from emails/documents: dates, amounts, merchant names, item descriptions, delivery status
FairScore™ Engine	XGBoost + SHAP + pgvector	Scores evidence across 6 dimensions, produces confidence score, outcome recommendation, and per-feature weights
Explainability Engine	Claude API / GPT-4	Generates plain-English decision explanations grounded in FairScore™ structured output
Audit Log Service	PostgreSQL (append-only)	Immutable SHA-256 hash-chained log of every action, decision, evidence item, and API call
Notification Service	AWS SNS + SES	Real-time push and email notifications at every status change for both parties
Card Member Portal	React + Tailwind + WebSocket	Dispute filing, real-time status tracking, decision view, evidence supplement upload
Merchant Portal	React + Tailwind + WebSocket	Dispute notification, targeted evidence submission, deadline countdown, decision rationale
AMEX Ops Dashboard	React + Recharts	Human review queue, model monitoring, bulk audit log, fairness metrics

3. End-to-End Data Flow

3.1 Happy Path — Auto-Resolution

The following sequence describes the data flow for a dispute that meets the auto-resolution confidence threshold (≥ 65):

```
Card Member Files Dispute (Portal)                                    t = 0
  │
  ▼
Dispute Service — creates dispute record, assigns case ID             t+1s
  │——— async trigger ———▶
```

```
                          Evidence Autopilot                        t+5s
                            ├── Transaction DB: metadata pull
                            ├── Merchant webhook: request evidence
                            ├── NLP Parser: email/document parse
                            ├── AMEX Code Mapper: reason code map
                            └── pgvector: retrieve similar cases
                          Evidence Bundle returned                  t+25s
  ◀── bundle ──────────────
  │
  ▼
FairScore™ Engine — scores 6 dimensions, SHAP weights              t+27s
  ├── confidence ≥ 65: auto-resolve path
  └── confidence < 65: human review queue
  │
  ▼ (auto path)
Explainability Engine — LLM generates CM + merchant explanation     t+35s
  │
  ▼
Audit Log Service — records full decision chain (hash-chained)      t+36s
  │
  ▼
Notification Service — notifies both parties simultaneously         t+37s
  │
  ▼
Portal updates — real-time WebSocket push to CM and merchant        t+38s

Total auto-resolution time: ~38 seconds end-to-end
```

3.2 Human Review Path

When FairScore™ confidence falls below the configured threshold (default: 65), the case routes to human review:

- Case is placed in the AMEX Ops Human Review Queue with full evidence bundle pre-loaded
- FairScore™ output and per-feature SHAP weights are displayed to the analyst for context
- Analyst makes a final decision with optional override note — this is also logged immutably
- Post-resolution, the human decision is fed back as a labelled training example to improve FairScore™

3.3 Evidence Bundle Schema

The Evidence Autopilot returns a structured JSON bundle consumed by FairScore™:

```
{
  "dispute_id": "DIS-2891",
  "filed_at": "2025-07-14T10:12:44Z",
  "transaction": {
    "amount": 15240, "currency": "INR",
    "merchant_id": "ZAR-BLR-042", "mcc": "5621",
    "timestamp": "2025-07-12T14:32:00Z",
    "card_present": true, "terminal_id": "ZAR-BLR-042"
  },
  "reason_code": { "code": "4513", "label": "Item Not As Described",
                   "confidence": 0.91, "required_evidence": ["receipt",
"delivery_confirm"] },
  "nlp_parsed": {
    "email_amount": 12800, "email_items": 3,
    "amount_mismatch": true, "mismatch_delta": 2440,
    "delivery_status": "no_confirmation_found"
  },
  "card_member_history": { "tenure_years": 7, "dispute_count_90d": 1, "dispute_freq":
"low" },
```

```
  "merchant_history": { "dispute_count_30d": 14, "win_rate": 0.43, "flag":
"moderate_risk" },
  "precedents": [{ "similarity": 0.94, "outcome": "favour_cm", "reason_code": "4513" },
...],
  "policy_check": { "within_dispute_window": true, "below_amount_cap": true }
}
```


4. FairScore™ Engine — Model Design

4.1 Feature Set (6 Dimensions)

FairScore™ evaluates each dispute across six independently scored dimensions. Each dimension contributes a weighted score (0–100) to the final confidence output:

Dimension	Weight	Description	Key Signal
1. Evidence Strength	25%	Completeness and quality of documentation from each side. Scored separately for card member and merchant, then differenced.	Receipt present? Signed? Timestamped? Delivery confirmation hash-matched?
2. Reason Code Alignment	20%	How well the dispute claim matches the evidentiary requirements of the mapped chargeback code.	AMEX Code 4513 requires receipt + delivery confirm. Missing items penalise merchant score.
3. Merchant History	15%	Merchant's historical dispute win/loss rate and dispute frequency. Serial dispute patterns are flagged.	>10 disputes/30d = moderate risk flag. >25 = high risk. Affects priors.
4. Card Member History	15%	Card member's dispute frequency and relationship tenure. Prevents bad-faith claim patterns.	First dispute in 12mo = low risk. 3+ disputes = frequency flag applied.
5. Case Precedent (RAG)	15%	Similarity score to past resolved cases retrieved via pgvector. Weighted by outcome and recency.	Top-k=20 similar cases retrieved. Outcome distribution used as Bayesian prior.
6. Policy Compliance	10%	Whether transaction falls within dispute window, amount thresholds, and MCC-specific policy rules.	Dispute must be filed within 120 days. Amount must be within AMEX policy caps.

4.2 Model Architecture

FairScore™ uses an XGBoost gradient boosting classifier trained on historical dispute outcomes, with SHAP (SHapley Additive exPlanations) values used for per-feature explainability:

- **Model type:** XGBoost binary classifier (favour_cm vs favour_merchant)
- **Training data:** Synthetic dispute outcomes generated from AMEX chargeback code decision trees + historical pattern augmentation
- **Output:** Probability score (0.0–1.0) converted to confidence (0–100), per-feature SHAP values, recommended outcome
- **Threshold:** Confidence ≥ 65 → auto-resolve. Confidence 50–64 → human review. Confidence < 50 → escalate with full context
- **SHAP integration:** TreeExplainer produces signed per-feature contributions used directly by Explainability Engine as structured input

4.3 RAG Precedent Retrieval

Past resolved disputes are stored as vector embeddings in a pgvector index on PostgreSQL. At inference time, the top-20 most similar cases are retrieved and their outcome distribution is used as a Bayesian prior:

```
# Retrieval query (pseudocode)
embedding = embed(dispute_features)  # sentence-transformers/all-MiniLM-L6-v2
similar_cases = pgvector.query(
    vector=embedding,
    top_k=20,
    filter={"reason_code": dispute.reason_code, "mcc": transaction.mcc}
)
precedent_score = weighted_outcome_distribution(similar_cases, recency_decay=0.95)
```

> Key insight: Using case precedent as a first-class input (not just a post-hoc reference) means FairScore™ decisions are consistent over time and get better as the resolved case base grows. This is the RAG-based learning loop.

5. Explainability Engine

5.1 Design Philosophy

The Explainability Engine does not generate post-hoc rationalisations. Explanations are structurally grounded in FairScore™ output — every sentence in the explanation maps directly to a SHAP feature contribution or precedent statistic. This makes explanations both accurate and defensible.

> Regulatory relevance: EU AI Act Article 22 and CFPB guidance both require that automated financial decisions be explainable to affected parties. ResolveIQ's Explainability Engine is designed to satisfy these requirements out of the box.

5.2 Prompt Architecture

The LLM (Claude API / GPT-4) receives a structured JSON prompt derived from FairScore™ output. It is never given raw evidence to interpret — only the structured scoring output and SHAP values:

```
SYSTEM PROMPT:
You are a financial decision explainer. Generate a clear, factual explanation of a
dispute decision for the [card_member / merchant]. Use only the structured data
provided.
Do not add information not present in the input. Use plain English. Be specific.

USER PROMPT (structured JSON):
{
  "outcome": "favour_card_member",
  "confidence": 91,
  "shap_contributions": {
    "evidence_strength_merchant": -0.34,   // negative = hurts merchant
    "reason_code_alignment": +0.28,
    "precedent_outcome_rate": +0.22,
    "card_member_history": +0.07,
    "policy_compliance": +0.09
  },
  "key_facts": {
    "merchant_delivery_confirmation": "not_provided",
    "amount_mismatch_delta": 2440,
    "precedent_cm_win_rate": 0.91,
    "reason_code": "4513"
  },
  "audience": "card_member"   // generates card_member-facing version
```

```
}
```

5.3 Output Format

The engine generates two distinct explanations per decision — one for the card member and one for the merchant — using audience-appropriate language:

> Card member example: "We resolved this dispute in your favour because: (1) the merchant was unable to provide a signed delivery confirmation within the required 72-hour window, (2) the transaction amount (₹15,240) does not match the amount on your order confirmation (₹12,800), and (3) 91% of similar cases in our system resolved in the card member's favour. This decision was made in accordance with AMEX Chargeback Code 4513."

> Merchant example: "We were unable to resolve this in your favour because: (1) you did not provide a signed delivery confirmation within the required 72-hour window — this was the highest-weight missing item, (2) the charge amount (₹15,240) could not be reconciled with the card member's order confirmation (₹12,800). We recommend reviewing your order fulfilment timestamps and ensuring receipt line items match transaction records for future disputes."

6. Key API Contracts

6.1 Dispute Filing API

```
POST /api/v1/disputes
Authorization: Bearer {jwt_token}

Request body:
{
  "transaction_id": "TXN-20250714-88291",
  "reason_code": "4513",
  "card_member_statement": "Item received was different from what I ordered",
  "supplementary_evidence": []  // optional file upload references
}

Response 202 Accepted:
{
  "dispute_id": "DIS-2891",
  "status": "evidence_collection",
  "estimated_resolution": "5m",
  "websocket_channel": "ws://api.resolveiq.com/ws/DIS-2891"
}
```

6.2 Merchant Evidence Webhook

```
POST /api/v1/merchant/evidence/{dispute_id}
Authorization: Bearer {merchant_api_key}

Request body:
{
  "delivery_confirmation": { "signed": true, "timestamp": "2025-07-12T16:00:00Z",
"hash": "sha256:..." },
  "receipt": { "line_items": [...], "total": 15240, "currency": "INR" },
  "return_policy": "15 days",
  "merchant_statement": "Item was delivered as described"
}
```

```
Response 200 OK:
{
  "evidence_received": true,
  "fairscore_rerun_triggered": true,
  "deadline_extended": false
}
```

6.3 FairScore™ Internal API

```
POST /internal/v1/fairscore
X-Service-Key: {internal_service_key}

Request: Evidence Bundle (see §3.3)

Response:
{
  "confidence": 91,
  "outcome": "favour_card_member",
  "dimension_scores": {
    "evidence_strength": { "cm": 88, "merchant": 18 },
    "reason_code_alignment": 91,
    "merchant_history": 62,
    "cm_history": 85,
    "case_precedent": 91,
    "policy_compliance": 100
  },
  "shap_values": { "evidence_strength_merchant": -0.34, ... },
  "auto_resolve": true,
  "routing": "auto_resolve"  // or "human_review"
}
```

7. Security Architecture

7.1 Authentication & Authorisation

Principal	Auth Mechanism	Identity Source	Access Scope
Card Member Portal	JWT (RS256)	AMEX SSO / OAuth 2.0	Scoped to own dispute records only
Merchant Portal	API Key + JWT	Merchant onboarding portal	Scoped to own merchant ID disputes
AMEX Ops Dashboard	MFA + JWT	AMEX internal SSO (SAML)	Role-based: Analyst / Supervisor / Read-only
Internal Services	mTLS + Service Key	AWS IAM + Secrets Manager	Least-privilege per service
Merchant Webhooks	HMAC-SHA256 signature	Shared secret per merchant	Replay-attack prevention via timestamp nonce

7.2 Data Protection

- **Data in transit:** TLS 1.3 enforced on all external and internal service communication
- **Data at rest:** AES-256 encryption on all PostgreSQL volumes (AWS RDS with encryption enabled)
- **PII handling:** Card member names, transaction details, and email content are tokenised before storage; raw PII retained in isolated vault

- **LLM data handling:** No raw PII sent to LLM APIs; all prompts use tokenised references resolved server-side
- **Audit log integrity:** Append-only PostgreSQL with row-level SHA-256 hash chaining; hash of record n-1 is embedded in record n; tampering is detectable

7.3 Audit Log Hash Chain Schema

```
CREATE TABLE audit_log (
  id            BIGSERIAL PRIMARY KEY,
  dispute_id    TEXT NOT NULL,
  actor         TEXT NOT NULL,          -- SYSTEM | CARD_MEMBER | MERCHANT |
OPS_ANALYST
  action        TEXT NOT NULL,
  payload       JSONB,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  prev_hash     TEXT,                   -- SHA-256 of previous record
  record_hash   TEXT NOT NULL           -- SHA-256(id || actor || action || payload ||
prev_hash)
);

-- Append-only enforced via row-level security policy:
ALTER TABLE audit_log ENABLE ROW LEVEL SECURITY;
CREATE POLICY no_update_delete ON audit_log FOR UPDATE USING (false);
CREATE POLICY no_delete ON audit_log FOR DELETE USING (false);
```

8. Infrastructure & Scalability

8.1 AWS Infrastructure Stack

Layer	Service	Role
Compute	AWS Lambda	All FastAPI services deployed as Lambda functions via Mangum adapter. Zero-infrastructure scaling.
API Layer	AWS API Gateway	REST API gateway with JWT authoriser, rate limiting, and WAF integration.
Object Storage	AWS S3	Evidence files (receipts, documents) stored with server-side AES-256 encryption and lifecycle policies.
Database	AWS RDS (PostgreSQL)	Primary data store for disputes, decisions, card member history. Multi-AZ for HA. pgvector extension for RAG.
Caching	AWS ElastiCache (Redis)	FairScore™ inference cache for duplicate submissions. Session state for portal WebSockets.
Messaging	AWS SQS + SNS	Async event queue between Dispute Service and Evidence Autopilot. SNS fan-out for notifications.
Email	AWS SES	Transactional email for dispute notifications, decision deliveries, deadline reminders.
Monitoring	Prometheus + Grafana	Model drift detection, API latency, fairness metric dashboards, SLA alerting.
Secrets	AWS Secrets Manager	LLM API keys, merchant webhook secrets, internal service keys. Rotated automatically.
CDN	AWS CloudFront	Portal static assets served globally. Edge caching for public API responses.

8.2 Scalability Targets

Component	Target SLA	Notes
FairScore™ inference latency	< 200ms per case	Parallelisable across thousands of concurrent disputes
pgvector precedent retrieval	< 100ms for top-20	HNSW index on embeddings; millions of cases supported
Evidence Autopilot	< 30s end-to-end	Parallel source collection; fails gracefully if any source unavailable
LLM explanation generation	< 10s	Async; streamed to portal via WebSocket
Portal WebSocket updates	Real-time (< 500ms)	AWS API Gateway WebSocket + Redis pub/sub
Auto-resolution throughput	> 1,000 cases/minute	Lambda concurrency limit configurable; horizontally unbounded
Audit log write latency	< 50ms per record	Async write; non-blocking on resolution path
Database storage	Petabyte-scale	RDS with S3 overflow for long-term audit archival

8.3 Monitoring & Alerting

All services emit structured logs and metrics to Prometheus. Key alert conditions:

- FairScore™ confidence distribution shift > 5% from 30-day baseline → model drift alert
- Auto-resolution rate drops below 60% → pipeline health alert
- Audit log hash chain integrity check failure → critical security alert (PagerDuty)
- LLM API latency > 15s → fallback to template-based explanation
- Any service error rate > 1% → oncall alert

9. Technology Stack Summary

Layer	Technology	Purpose	Deployment
Frontend	React 18 + Tailwind CSS + WebSocket API	Card Member, Merchant, and AMEX Ops portals	Vite + AWS CloudFront
Backend API	Python 3.11 + FastAPI + Mangum	All core services	AWS Lambda + API Gateway
NLP / AI	spaCy 3.7 + Hugging Face DistilBERT	Evidence parsing from emails and documents	AWS Lambda (GPU optional)
Decision Model	XGBoost 2.0 + SHAP 0.44	FairScore™ weighing engine with explainability	AWS Lambda
Vector DB	pgvector 0.7 on PostgreSQL 16	RAG-based case similarity retrieval	AWS RDS
Embeddings	sentence-transformers/all-MiniLM-L6-v2	Dispute embedding generation	Hugging Face Inference
LLM Layer	Claude API (claude-sonnet) / GPT-4o	Plain-English decision explanation generation	External API (no PII)

Layer	Technology	Purpose	Deployment
Primary Database	PostgreSQL 16 (AWS RDS)	Disputes, decisions, audit log, card member data	Multi-AZ, encrypted
Cache	Redis 7 (AWS ElastiCache)	Inference cache, WebSocket state	In-memory
Messaging	AWS SQS + SNS	Async evidence collection triggers, notifications	Managed, serverless
Observability	Prometheus + Grafana	Model monitoring, latency, fairness metrics	Self-hosted on ECS
Infrastructure	AWS (Lambda, RDS, S3, API GW, SES, CloudFront)	Full cloud deployment	Terraform IaC

10. Implementation Roadmap

Phase	Timeline	Name	Deliverables
Phase 1	Weeks 1–3	Foundation	Core dispute filing API · Transaction evidence collection · FairScore™ v1 (rule-based, no ML) · Card member + merchant portals MVP · Basic audit log
Phase 2	Weeks 4–6	AI Layer	NLP evidence parser (spaCy + DistilBERT) · XGBoost FairScore™ model (trained on synthetic data) · pgvector precedent engine · SHAP integration
Phase 3	Weeks 7–8	Explainability	LLM-powered explanation generation · Dual-audience explanation (CM + merchant) · Hash-chained audit log · Appeal flow
Phase 4	Weeks 9–10	Ops & Polish	AMEX Ops dashboard · Model drift monitoring (Prometheus + Grafana) · Fairness metrics (Gini coefficient) · End-to-end load testing and optimisation

> Feasibility note: All components (spaCy, XGBoost, pgvector, SHAP, FastAPI, Lambda) are mature, production-ready open-source libraries. No research-level AI is required. The system can be deployed in a safe human-in-the-loop configuration from Day 1, with the AI layer progressively taking over as confidence thresholds are validated.

11. Success Metrics & KPIs

< 5 min	≥ 70%	40%	100%
Auto-resolution time	Auto-resolution rate	Appeal rate reduction	Audit completeness

Metric	Baseline	Target	Measurement Method
Dispute Resolution Time	14–45 days (current)	< 5 minutes (auto-resolvable cases)	Dispute Service timestamp delta

Metric	Baseline	Target	Measurement Method
Auto-Resolution Rate	0% (all manual)	≥ 70% of inbound disputes	FairScore™ routing logs
Appeal Rate	Baseline (TBD)	≥ 40% reduction	Dispute lifecycle tracking
Card Member CSAT	Baseline (TBD)	+15 point improvement on dispute CSAT	Post-resolution survey
Merchant Satisfaction	Baseline (TBD)	≥ 30% improvement on transparency survey	Merchant portal feedback
Audit Completeness	0% (no audit trail)	100% of decisions have immutable audit record	Audit log coverage check
Model Fairness (Gini)	N/A	Decision parity within ±5% across demographics	Prometheus fairness dashboard
FairScore™ Accuracy	N/A (new)	≥ 85% alignment with expert human decisions	Back-testing on historical cases
Ops Cost per Dispute	$25–$50 (manual)	< $5 per auto-resolved case	AWS cost allocation + ops hours

12. Conclusion

ResolveIQ's technical architecture is designed around three core commitments: speed, fairness, and transparency. Every component decision — from XGBoost over black-box neural networks, to hash-chained audit logs over conventional logging, to structured LLM prompts over free-form generation — reflects these values.

The architecture is modular, feasible with off-the-shelf components, and production-ready in a phased 10-week build. It scales horizontally on AWS serverless infrastructure, degrades gracefully under data source failures, and improves continuously through its RAG-based learning loop.

> **ResolveIQ is not just a faster dispute tool. It is the technical foundation of a trust layer between card members, merchants, and the issuer — built on AI that is fair, explainable, and auditable by design.**